

Advent Of Code Solutions

1

Generated by Doxygen 1.9.8

1 My solutions to the Advent Of Code problems.	1
1.1 Dependencies	1
1.1.1 Build Project	1
1.1.2 Documentation	1
1.1.3 License	2
2 GNU GENERAL PUBLIC LICENSE	3
2.1 Preamble	3
2.2 TERMS AND CONDITIONS	4
2.2.1 0. Definitions.	4
2.2.2 1. Source Code.	5
2.2.3 2. Basic Permissions.	5
2.2.4 3. Protecting Users' Legal Rights From Anti-Circumvention Law.	5
2.2.5 4. Conveying Verbatim Copies.	6
2.2.6 5. Conveying Modified Source Versions.	6
2.2.7 6. Conveying Non-Source Forms.	6
2.2.8 7. Additional Terms.	8
2.2.9 8. Termination.	8
2.2.10 9. Acceptance Not Required for Having Copies.	9
2.2.11 10. Automatic Licensing of Downstream Recipients.	9
2.2.12 11. Patents.	9
2.2.13 12. No Surrender of Others' Freedom.	10
2.2.14 13. Use with the GNU Affero General Public License.	10
2.2.15 14. Revised Versions of this License.	10
2.2.16 15. Disclaimer of Warranty.	11
2.2.17 16. Limitation of Liability.	11
2.2.18 17. Interpretation of Sections 15 and 16.	11
2.3 How to Apply These Terms to Your New Programs	11
3 Todo List	13
4 Class Index	15
4.1 Class List	15
5 File Index	17
5.1 File List	17
6 Class Documentation	19
6.1 histogram_lock< max_elem, min_elem, T > Class Template Reference	19
6.1.1 Detailed Description	19
6.1.2 Constructor & Destructor Documentation	20

6.1.2.1 histogram_lock()	20
6.1.3 Member Function Documentation	20
6.1.3.1 get_current_value()	20
6.1.3.2 get_frequency()	20
6.1.3.3 operator+=()	21
6.1.3.4 operator-=()	21
6.1.3.5 operator==()	22
6.1.3.6 set_current_value()	22
6.2 input_parser< T > Class Template Reference	23
6.2.1 Detailed Description	23
6.2.2 Constructor & Destructor Documentation	23
6.2.2.1 input_parser()	23
6.2.3 Member Function Documentation	23
6.2.3.1 begin()	23
6.2.3.2 end()	24
6.3 input_parser< T >::itor Struct Reference	24
6.3.1 Detailed Description	24
6.3.2 Member Typedef Documentation	24
6.3.2.1 difference_type	24
6.3.2.2 iterator_category	25
6.3.2.3 value_type	25
6.3.3 Constructor & Destructor Documentation	25
6.3.3.1 itor() [1/2]	25
6.3.3.2 itor() [2/2]	25
6.3.4 Member Function Documentation	25
6.3.4.1 operator!=(())	25
6.3.4.2 operator*()	26
6.3.4.3 operator++() [1/2]	26
6.3.4.4 operator++() [2/2]	26
6.3.4.5 operator==(())	26
7 File Documentation	27
7.1 2025/Day1/input_parser.hpp File Reference	27
7.1.1 Detailed Description	28
7.2 input_parser.hpp	29
7.3 2025/Day1/input_parser.hxx File Reference	29
7.3.1 Detailed Description	30
7.4 input_parser.hxx	31
7.5 2025/Day1/input_parser_tests.cpp File Reference	32

7.5.1 Detailed Description	33
7.5.2 Function Documentation	34
7.5.2.1 TEST_CASE()	34
7.6 input_parser_tests.cpp	34
7.7 2025/Day1/lock.hpp File Reference	35
7.7.1 Detailed Description	36
7.8 lock.hpp	36
7.9 2025/Day1/lock_test.cpp File Reference	38
7.9.1 Detailed Description	39
7.9.2 Function Documentation	39
7.9.2.1 TEST_CASE() [1/3]	39
7.9.2.2 TEST_CASE() [2/3]	40
7.9.2.3 TEST_CASE() [3/3]	40
7.10 lock_test.cpp	40
7.11 2025/Day1/main.cpp File Reference	41
7.11.1 Detailed Description	42
7.11.2 Function Documentation	43
7.11.2.1 main()	43
7.12 main.cpp	43
Index	45

Chapter 1

My solutions to the Advent Of Code problems.

[Link to the Advent Of Code](#)

If you are also doing the problems, I encourage you to solve them independently to grow your own skills. Please do not use my solutions to cheat yourself of learning.

If you need to contact me, you can [email me](#).

1.1 Dependencies

- C++ 20
- Cmake - Build System
- Catch2 - Testing framework (Cmake will auto-install)

1.1.1 Build Project

Run: `cmake --preset Development -B build/` once from the project root to initialize the build system. See the Cmake documentation if this does not work.

Run `cmake --build build/` to build the project. The first build will be long, but subsequent builds should be faster.

Final executables will be in `build/<year>/Day<day>/`

1.1.2 Documentation

Install `doxygen` and run it in the main directory to generate HTML docs (`docs/html/index.html`), and a makefile for PDF documentation.

1.1.3 License

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

A copy of the license can be found here: [GNU GENERAL PUBLIC LICENSE](#)

Chapter 2

GNU GENERAL PUBLIC LICENSE

Version 3, 29 June 2007

Copyright (C) 2007 Free Software Foundation, Inc. <https://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

2.1 Preamble

The GNU General Public License is a free, copyleft license for software and other kinds of works.

The licenses for most software and other practical works are designed to take away your freedom to share and change the works. By contrast, the GNU General Public License is intended to guarantee your freedom to share and change all versions of a program—to make sure it remains free software for all its users. We, the Free Software Foundation, use the GNU General Public License for most of our software; it applies also to any other work released this way by its authors. You can apply it to your programs, too.

When we speak of free software, we are referring to freedom, not price. Our General Public Licenses are designed to make sure that you have the freedom to distribute copies of free software (and charge for them if you wish), that you receive source code or can get it if you want it, that you can change the software or use pieces of it in new free programs, and that you know you can do these things.

To protect your rights, we need to prevent others from denying you these rights or asking you to surrender the rights. Therefore, you have certain responsibilities if you distribute copies of the software, or if you modify it: responsibilities to respect the freedom of others.

For example, if you distribute copies of such a program, whether gratis or for a fee, you must pass on to the recipients the same freedoms that you received. You must make sure that they, too, receive or can get the source code. And you must show them these terms so they know their rights.

Developers that use the GNU GPL protect your rights with two steps: (1) assert copyright on the software, and (2) offer you this License giving you legal permission to copy, distribute and/or modify it.

For the developers' and authors' protection, the GPL clearly explains that there is no warranty for this free software. For both users' and authors' sake, the GPL requires that modified versions be marked as changed, so that their problems will not be attributed erroneously to authors of previous versions.

Some devices are designed to deny users access to install or run modified versions of the software inside them, although the manufacturer can do so. This is fundamentally incompatible with the aim of protecting users' freedom to change the software. The systematic pattern of such abuse occurs in the area of products for individuals to use, which is precisely where it is most unacceptable. Therefore, we have designed this version of the GPL to prohibit the practice for those products. If such problems arise substantially in other domains, we stand ready to extend this provision to those domains in future versions of the GPL, as needed to protect the freedom of users.

Finally, every program is threatened constantly by software patents. States should not allow patents to restrict development and use of software on general-purpose computers, but in those that do, we wish to avoid the special danger that patents applied to a free program could make it effectively proprietary. To prevent this, the GPL assures that patents cannot be used to render the program non-free.

The precise terms and conditions for copying, distribution and modification follow.

2.2 TERMS AND CONDITIONS

2.2.1 0. Definitions.

"This License" refers to version 3 of the GNU General Public License.

"Copyright" also means copyright-like laws that apply to other kinds of works, such as semiconductor masks.

"The Program" refers to any copyrightable work licensed under this License. Each licensee is addressed as "you". "Licensees" and "recipients" may be individuals or organizations.

To "modify" a work means to copy from or adapt all or part of the work in a fashion requiring copyright permission, other than the making of an exact copy. The resulting work is called a "modified version" of the earlier work or a work "based on" the earlier work.

A "covered work" means either the unmodified Program or a work based on the Program.

To "propagate" a work means to do anything with it that, without permission, would make you directly or secondarily liable for infringement under applicable copyright law, except executing it on a computer or modifying a private copy. Propagation includes copying, distribution (with or without modification), making available to the public, and in some countries other activities as well.

To "convey" a work means any kind of propagation that enables other parties to make or receive copies. Mere interaction with a user through a computer network, with no transfer of a copy, is not conveying.

An interactive user interface displays "Appropriate Legal Notices" to the extent that it includes a convenient and prominently visible feature that (1) displays an appropriate copyright notice, and (2) tells the user that there is no warranty for the work (except to the extent that warranties are provided), that licensees may convey the work under this License, and how to view a copy of this License. If the interface presents a list of user commands or options, such as a menu, a prominent item in the list meets this criterion.

2.2.2 1. Source Code.

The "source code" for a work means the preferred form of the work for making modifications to it. "Object code" means any non-source form of a work.

A "Standard Interface" means an interface that either is an official standard defined by a recognized standards body, or, in the case of interfaces specified for a particular programming language, one that is widely used among developers working in that language.

The "System Libraries" of an executable work include anything, other than the work as a whole, that (a) is included in the normal form of packaging a Major Component, but which is not part of that Major Component, and (b) serves only to enable use of the work with that Major Component, or to implement a Standard Interface for which an implementation is available to the public in source code form. A "Major Component", in this context, means a major essential component (kernel, window system, and so on) of the specific operating system (if any) on which the executable work runs, or a compiler used to produce the work, or an object code interpreter used to run it.

The "Corresponding Source" for a work in object code form means all the source code needed to generate, install, and (for an executable work) run the object code and to modify the work, including scripts to control those activities. However, it does not include the work's System Libraries, or general-purpose tools or generally available free programs which are used unmodified in performing those activities but which are not part of the work. For example, Corresponding Source includes interface definition files associated with source files for the work, and the source code for shared libraries and dynamically linked subprograms that the work is specifically designed to require, such as by intimate data communication or control flow between those subprograms and other parts of the work.

The Corresponding Source need not include anything that users can regenerate automatically from other parts of the Corresponding Source.

The Corresponding Source for a work in source code form is that same work.

2.2.3 2. Basic Permissions.

All rights granted under this License are granted for the term of copyright on the Program, and are irrevocable provided the stated conditions are met. This License explicitly affirms your unlimited permission to run the unmodified Program. The output from running a covered work is covered by this License only if the output, given its content, constitutes a covered work. This License acknowledges your rights of fair use or other equivalent, as provided by copyright law.

You may make, run and propagate covered works that you do not convey, without conditions so long as your license otherwise remains in force. You may convey covered works to others for the sole purpose of having them make modifications exclusively for you, or provide you with facilities for running those works, provided that you comply with the terms of this License in conveying all material for which you do not control copyright. Those thus making or running the covered works for you must do so exclusively on your behalf, under your direction and control, on terms that prohibit them from making any copies of your copyrighted material outside their relationship with you.

Conveying under any other circumstances is permitted solely under the conditions stated below. Sublicensing is not allowed; section 10 makes it unnecessary.

2.2.4 3. Protecting Users' Legal Rights From Anti-Circumvention Law.

No covered work shall be deemed part of an effective technological measure under any applicable law fulfilling obligations under article 11 of the WIPO copyright treaty adopted on 20 December 1996, or similar laws prohibiting or restricting circumvention of such measures.

When you convey a covered work, you waive any legal power to forbid circumvention of technological measures to the extent such circumvention is effected by exercising rights under this License with respect to the covered work, and you disclaim any intention to limit operation or modification of the work as a means of enforcing, against the work's users, your or third parties' legal rights to forbid circumvention of technological measures.

2.2.5 4. Conveying Verbatim Copies.

You may convey verbatim copies of the Program's source code as you receive it, in any medium, provided that you conspicuously and appropriately publish on each copy an appropriate copyright notice; keep intact all notices stating that this License and any non-permissive terms added in accord with section 7 apply to the code; keep intact all notices of the absence of any warranty; and give all recipients a copy of this License along with the Program.

You may charge any price or no price for each copy that you convey, and you may offer support or warranty protection for a fee.

2.2.6 5. Conveying Modified Source Versions.

You may convey a work based on the Program, or the modifications to produce it from the Program, in the form of source code under the terms of section 4, provided that you also meet all of these conditions:

- a) The work must carry prominent notices stating that you modified it, and giving a relevant date.
- b) The work must carry prominent notices stating that it is released under this License and any conditions added under section 7. This requirement modifies the requirement in section 4 to "keep intact all notices".
- c) You must license the entire work, as a whole, under this License to anyone who comes into possession of a copy. This License will therefore apply, along with any applicable section 7 additional terms, to the whole of the work, and all its parts, regardless of how they are packaged. This License gives no permission to license the work in any other way, but it does not invalidate such permission if you have separately received it.
- d) If the work has interactive user interfaces, each must display Appropriate Legal Notices; however, if the Program has interactive interfaces that do not display Appropriate Legal Notices, your work need not make them do so.

A compilation of a covered work with other separate and independent works, which are not by their nature extensions of the covered work, and which are not combined with it such as to form a larger program, in or on a volume of a storage or distribution medium, is called an "aggregate" if the compilation and its resulting copyright are not used to limit the access or legal rights of the compilation's users beyond what the individual works permit. Inclusion of a covered work in an aggregate does not cause this License to apply to the other parts of the aggregate.

2.2.7 6. Conveying Non-Source Forms.

You may convey a covered work in object code form under the terms of sections 4 and 5, provided that you also convey the machine-readable Corresponding Source under the terms of this License, in one of these ways:

- a) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by the Corresponding Source fixed on a durable physical medium customarily used for software interchange.
- b) Convey the object code in, or embodied in, a physical product (including a physical distribution medium), accompanied by a written offer, valid for at least three years and valid for as long as you offer spare parts or customer support for that product model, to give anyone who possesses the object code either (1) a copy of the Corresponding Source for all the software in the product that is covered by this License, on a durable physical medium customarily used for software interchange, for a price no more than your reasonable cost of physically performing this conveying of source, or (2) access to copy the Corresponding Source from a network server at no charge.

- c) Convey individual copies of the object code with a copy of the written offer to provide the Corresponding Source. This alternative is allowed only occasionally and noncommercially, and only if you received the object code with such an offer, in accord with subsection 6b.
- d) Convey the object code by offering access from a designated place (gratis or for a charge), and offer equivalent access to the Corresponding Source in the same way through the same place at no further charge. You need not require recipients to copy the Corresponding Source along with the object code. If the place to copy the object code is a network server, the Corresponding Source may be on a different server (operated by you or a third party) that supports equivalent copying facilities, provided you maintain clear directions next to the object code saying where to find the Corresponding Source. Regardless of what server hosts the Corresponding Source, you remain obligated to ensure that it is available for as long as needed to satisfy these requirements.
- e) Convey the object code using peer-to-peer transmission, provided you inform other peers where the object code and Corresponding Source of the work are being offered to the general public at no charge under subsection 6d.

A separable portion of the object code, whose source code is excluded from the Corresponding Source as a System Library, need not be included in conveying the object code work.

A "User Product" is either (1) a "consumer product", which means any tangible personal property which is normally used for personal, family, or household purposes, or (2) anything designed or sold for incorporation into a dwelling. In determining whether a product is a consumer product, doubtful cases shall be resolved in favor of coverage. For a particular product received by a particular user, "normally used" refers to a typical or common use of that class of product, regardless of the status of the particular user or of the way in which the particular user actually uses, or expects or is expected to use, the product. A product is a consumer product regardless of whether the product has substantial commercial, industrial or non-consumer uses, unless such uses represent the only significant mode of use of the product.

"Installation Information" for a User Product means any methods, procedures, authorization keys, or other information required to install and execute modified versions of a covered work in that User Product from a modified version of its Corresponding Source. The information must suffice to ensure that the continued functioning of the modified object code is in no case prevented or interfered with solely because modification has been made.

If you convey an object code work under this section in, or with, or specifically for use in, a User Product, and the conveying occurs as part of a transaction in which the right of possession and use of the User Product is transferred to the recipient in perpetuity or for a fixed term (regardless of how the transaction is characterized), the Corresponding Source conveyed under this section must be accompanied by the Installation Information. But this requirement does not apply if neither you nor any third party retains the ability to install modified object code on the User Product (for example, the work has been installed in ROM).

The requirement to provide Installation Information does not include a requirement to continue to provide support service, warranty, or updates for a work that has been modified or installed by the recipient, or for the User Product in which it has been modified or installed. Access to a network may be denied when the modification itself materially and adversely affects the operation of the network or violates the rules and protocols for communication across the network.

Corresponding Source conveyed, and Installation Information provided, in accord with this section must be in a format that is publicly documented (and with an implementation available to the public in source code form), and must require no special password or key for unpacking, reading or copying.

2.2.8 7. Additional Terms.

"Additional permissions" are terms that supplement the terms of this License by making exceptions from one or more of its conditions. Additional permissions that are applicable to the entire Program shall be treated as though they were included in this License, to the extent that they are valid under applicable law. If additional permissions apply only to part of the Program, that part may be used separately under those permissions, but the entire Program remains governed by this License without regard to the additional permissions.

When you convey a copy of a covered work, you may at your option remove any additional permissions from that copy, or from any part of it. (Additional permissions may be written to require their own removal in certain cases when you modify the work.) You may place additional permissions on material, added by you to a covered work, for which you have or can give appropriate copyright permission.

Notwithstanding any other provision of this License, for material you add to a covered work, you may (if authorized by the copyright holders of that material) supplement the terms of this License with terms:

- a) Disclaiming warranty or limiting liability differently from the terms of sections 15 and 16 of this License; or
- b) Requiring preservation of specified reasonable legal notices or author attributions in that material or in the Appropriate Legal Notices displayed by works containing it; or
- c) Prohibiting misrepresentation of the origin of that material, or requiring that modified versions of such material be marked in reasonable ways as different from the original version; or
- d) Limiting the use for publicity purposes of names of licensors or authors of the material; or
- e) Declining to grant rights under trademark law for use of some trade names, trademarks, or service marks; or
- f) Requiring indemnification of licensors and authors of that material by anyone who conveys the material (or modified versions of it) with contractual assumptions of liability to the recipient, for any liability that these contractual assumptions directly impose on those licensors and authors.

All other non-permissive additional terms are considered "further restrictions" within the meaning of section 10. If the Program as you received it, or any part of it, contains a notice stating that it is governed by this License along with a term that is a further restriction, you may remove that term. If a license document contains a further restriction but permits relicensing or conveying under this License, you may add to a covered work material governed by the terms of that license document, provided that the further restriction does not survive such relicensing or conveying.

If you add terms to a covered work in accord with this section, you must place, in the relevant source files, a statement of the additional terms that apply to those files, or a notice indicating where to find the applicable terms.

Additional terms, permissive or non-permissive, may be stated in the form of a separately written license, or stated as exceptions; the above requirements apply either way.

2.2.9 8. Termination.

You may not propagate or modify a covered work except as expressly provided under this License. Any attempt otherwise to propagate or modify it is void, and will automatically terminate your rights under this License (including any patent licenses granted under the third paragraph of section 11).

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, you do not qualify to receive new licenses for the same material under section 10.

2.2.10 9. Acceptance Not Required for Having Copies.

You are not required to accept this License in order to receive or run a copy of the Program. Ancillary propagation of a covered work occurring solely as a consequence of using peer-to-peer transmission to receive a copy likewise does not require acceptance. However, nothing other than this License grants you permission to propagate or modify any covered work. These actions infringe copyright if you do not accept this License. Therefore, by modifying or propagating a covered work, you indicate your acceptance of this License to do so.

2.2.11 10. Automatic Licensing of Downstream Recipients.

Each time you convey a covered work, the recipient automatically receives a license from the original licensors, to run, modify and propagate that work, subject to this License. You are not responsible for enforcing compliance by third parties with this License.

An "entity transaction" is a transaction transferring control of an organization, or substantially all assets of one, or subdividing an organization, or merging organizations. If propagation of a covered work results from an entity transaction, each party to that transaction who receives a copy of the work also receives whatever licenses to the work the party's predecessor in interest had or could give under the previous paragraph, plus a right to possession of the Corresponding Source of the work from the predecessor in interest, if the predecessor has it or can get it with reasonable efforts.

You may not impose any further restrictions on the exercise of the rights granted or affirmed under this License. For example, you may not impose a license fee, royalty, or other charge for exercise of rights granted under this License, and you may not initiate litigation (including a cross-claim or counterclaim in a lawsuit) alleging that any patent claim is infringed by making, using, selling, offering for sale, or importing the Program or any portion of it.

2.2.12 11. Patents.

A "contributor" is a copyright holder who authorizes use under this License of the Program or a work on which the Program is based. The work thus licensed is called the contributor's "contributor version".

A contributor's "essential patent claims" are all patent claims owned or controlled by the contributor, whether already acquired or hereafter acquired, that would be infringed by some manner, permitted by this License, of making, using, or selling its contributor version, but do not include claims that would be infringed only as a consequence of further modification of the contributor version. For purposes of this definition, "control" includes the right to grant patent sublicenses in a manner consistent with the requirements of this License.

Each contributor grants you a non-exclusive, worldwide, royalty-free patent license under the contributor's essential patent claims, to make, use, sell, offer for sale, import and otherwise run, modify and propagate the contents of its contributor version.

In the following three paragraphs, a "patent license" is any express agreement or commitment, however denominated, not to enforce a patent (such as an express permission to practice a patent or covenant not to sue for patent infringement). To "grant" such a patent license to a party means to make such an agreement or commitment not to enforce a patent against the party.

If you convey a covered work, knowingly relying on a patent license, and the Corresponding Source of the work is not available for anyone to copy, free of charge and under the terms of this License, through a publicly available network server or other readily accessible means, then you must either (1) cause the Corresponding Source to be so available, or (2) arrange to deprive yourself of the benefit of the patent license for this particular work, or (3) arrange, in a manner consistent with the requirements of this License, to extend the patent license to downstream recipients. "Knowingly

relying" means you have actual knowledge that, but for the patent license, your conveying the covered work in a country, or your recipient's use of the covered work in a country, would infringe one or more identifiable patents in that country that you have reason to believe are valid.

If, pursuant to or in connection with a single transaction or arrangement, you convey, or propagate by procuring conveyance of, a covered work, and grant a patent license to some of the parties receiving the covered work authorizing them to use, propagate, modify or convey a specific copy of the covered work, then the patent license you grant is automatically extended to all recipients of the covered work and works based on it.

A patent license is "discriminatory" if it does not include within the scope of its coverage, prohibits the exercise of, or is conditioned on the non-exercise of one or more of the rights that are specifically granted under this License. You may not convey a covered work if you are a party to an arrangement with a third party that is in the business of distributing software, under which you make payment to the third party based on the extent of your activity of conveying the work, and under which the third party grants, to any of the parties who would receive the covered work from you, a discriminatory patent license (a) in connection with copies of the covered work conveyed by you (or copies made from those copies), or (b) primarily for and in connection with specific products or compilations that contain the covered work, unless you entered into that arrangement, or that patent license was granted, prior to 28 March 2007.

Nothing in this License shall be construed as excluding or limiting any implied license or other defenses to infringement that may otherwise be available to you under applicable patent law.

2.2.13 12. No Surrender of Others' Freedom.

If conditions are imposed on you (whether by court order, agreement or otherwise) that contradict the conditions of this License, they do not excuse you from the conditions of this License. If you cannot convey a covered work so as to satisfy simultaneously your obligations under this License and any other pertinent obligations, then as a consequence you may not convey it at all. For example, if you agree to terms that obligate you to collect a royalty for further conveying from those to whom you convey the Program, the only way you could satisfy both those terms and this License would be to refrain entirely from conveying the Program.

2.2.14 13. Use with the GNU Affero General Public License.

Notwithstanding any other provision of this License, you have permission to link or combine any covered work with a work licensed under version 3 of the GNU Affero General Public License into a single combined work, and to convey the resulting work. The terms of this License will continue to apply to the part which is the covered work, but the special requirements of the GNU Affero General Public License, section 13, concerning interaction through a network will apply to the combination as such.

2.2.15 14. Revised Versions of this License.

The Free Software Foundation may publish revised and/or new versions of the GNU General Public License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns.

Each version is given a distinguishing version number. If the Program specifies that a certain numbered version of the GNU General Public License "or any later version" applies to it, you have the option of following the terms and conditions either of that numbered version or of any later version published by the Free Software Foundation. If the Program does not specify a version number of the GNU General Public License, you may choose any version ever published by the Free Software Foundation.

If the Program specifies that a proxy can decide which future versions of the GNU General Public License can be used, that proxy's public statement of acceptance of a version permanently authorizes you to choose that version for the Program.

Later license versions may give you additional or different permissions. However, no additional obligations are imposed on any author or copyright holder as a result of your choosing to follow a later version.

2.2.16 15. Disclaimer of Warranty.

THERE IS NO WARRANTY FOR THE PROGRAM, TO THE EXTENT PERMITTED BY APPLICABLE LAW. EXCEPT WHEN OTHERWISE STATED IN WRITING THE COPYRIGHT HOLDERS AND/OR OTHER PARTIES PROVIDE THE PROGRAM "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESSED OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE. THE ENTIRE RISK AS TO THE QUALITY AND PERFORMANCE OF THE PROGRAM IS WITH YOU. SHOULD THE PROGRAM PROVE DEFECTIVE, YOU ASSUME THE COST OF ALL NECESSARY SERVICING, REPAIR OR CORRECTION.

2.2.17 16. Limitation of Liability.

IN NO EVENT UNLESS REQUIRED BY APPLICABLE LAW OR AGREED TO IN WRITING WILL ANY COPYRIGHT HOLDER, OR ANY OTHER PARTY WHO MODIFIES AND/OR CONVEYS THE PROGRAM AS PERMITTED ABOVE, BE LIABLE TO YOU FOR DAMAGES, INCLUDING ANY GENERAL, SPECIAL, INCIDENTAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF THE USE OR INABILITY TO USE THE PROGRAM (INCLUDING BUT NOT LIMITED TO LOSS OF DATA OR DATA BEING RENDERED INACCURATE OR LOSSES SUSTAINED BY YOU OR THIRD PARTIES OR A FAILURE OF THE PROGRAM TO OPERATE WITH ANY OTHER PROGRAMS), EVEN IF SUCH HOLDER OR OTHER PARTY HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

2.2.18 17. Interpretation of Sections 15 and 16.

If the disclaimer of warranty and limitation of liability provided above cannot be given local legal effect according to their terms, reviewing courts shall apply local law that most closely approximates an absolute waiver of all civil liability in connection with the Program, unless a warranty or assumption of liability accompanies a copy of the Program in return for a fee.

END OF TERMS AND CONDITIONS

2.3 How to Apply These Terms to Your New Programs

If you develop a new program, and you want it to be of the greatest possible use to the public, the best way to achieve this is to make it free software which everyone can redistribute and change under these terms.

To do so, attach the following notices to the program. It is safest to attach them to the start of each source file to most effectively state the exclusion of warranty; and each file should have at least the "copyright" line and a pointer to where the full notice is found.

```
<one line to give the program's name and a brief idea of what it does.>
Copyright (C) <year> <name of author>
```

```
This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.
```

```
This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
GNU General Public License for more details.
```

```
You should have received a copy of the GNU General Public License
along with this program. If not, see <https://www.gnu.org/licenses/>.
```

Also add information on how to contact you by electronic and paper mail.

If the program does terminal interaction, make it output a short notice like this when it starts in an interactive mode:

```
<program> Copyright (C) <year> <name of author>
This program comes with ABSOLUTELY NO WARRANTY; for details type `show w'.
This is free software, and you are welcome to redistribute it
under certain conditions; type `show c' for details.
```

The hypothetical commands `show w' and `show c' should show the appropriate parts of the General Public License. Of course, your program's commands might be different; for a GUI interface, you would use an "about box".

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see <https://www.gnu.org/licenses/>.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read <https://www.gnu.org/licenses/why-not-lgpl.html>.

Chapter 3

Todo List

Class `histogram_lock< max_elem, min_elem, T >`

refactor the histogram into another class, and have it a separate dependency from the lock

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

histogram_lock< max_elem, min_elem, T >	
A lock that keeps a histogram of how frequently each value is present	19
input_parser< T >	
An iterator that goes converts file to signed integers	23
input_parser< T >::itor	24

Chapter 5

File Index

5.1 File List

Here is a list of all documented files with brief descriptions:

2025/Day1/ input_parser.hpp	
Functions to parse the input of the file, converting them to signed integers	27
2025/Day1/ input_parser.hxx	
Implementation of the file parser iterator	29
2025/Day1/ input_parser_tests.cpp	
Tests the input parser	32
2025/Day1/ lock.hpp	
Implements a lock that holds a histogram of values	35
2025/Day1/ lock_test.cpp	
Tests the lock to see if it works as expected	38
2025/Day1/ main.cpp	
My Day One Solution Problem	41

Chapter 6

Class Documentation

6.1 histogram_lock< max_elem, min_elem, T > Class Template Reference

A lock that keeps a histogram of how frequently each value is present.

```
#include <lock.hpp>
```

Public Member Functions

- [histogram_lock](#) (T init)
Construct a new lock object at position init.
- void [set_current_value](#) (T val)
sets the current position of the lock
- void [operator==](#) (T val)
Sets the current position of the lock.
- void [operator+=](#) (T val)
Increases the position of the lock by val amount.
- void [operator-=](#) (T val)
Decreases the position of the lock by val amount.
- const T [get_current_value](#) () const
Get the current value object.
- T [get_frequency](#) (T val)
Gets how many times the lock was at val.

6.1.1 Detailed Description

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>  
requires (max_elem > min_elem)  
class histogram_lock< max_elem, min_elem, T >
```

A lock that keeps a histogram of how frequently each value is present.

Todo refactor the histogram into another class, and have it a separate dependency from the lock

Template Parameters

<i>max_elem</i>	The largest value on the lock
<i>min_elem</i>	The smallest value on the lock
<i>T</i>	The type of the values on the lock

Definition at line 46 of file [lock.hpp](#).

6.1.2 Constructor & Destructor Documentation

6.1.2.1 histogram_lock()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
histogram\_lock< max_elem, min_elem, T >::histogram\_lock (
    T init ) [inline]
```

Construct a new lock object at position *init*.

Parameters

<i>init</i>	Value that the lock starts at
-------------	-------------------------------

Definition at line 53 of file [lock.hpp](#).

6.1.3 Member Function Documentation

6.1.3.1 get_current_value()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
const T histogram\_lock< max_elem, min_elem, T >::get_current_value ( ) const [inline]
```

Get the current value object.

Definition at line 98 of file [lock.hpp](#).

6.1.3.2 get_frequency()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
T histogram\_lock< max_elem, min_elem, T >::get_frequency (
    T val ) [inline]
```

Gets how many times the lock was at *val*.

Parameters

<i>val</i>	
------------	--

Returns

count

Definition at line 106 of file [lock.hpp](#).

6.1.3.3 operator+=()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
void histogram_lock< max_elem, min_elem, T >::operator+= (
    T val ) [inline]
```

Increases the position of the lock by val amount.

Parameters

<i>val</i>	Delta
------------	-------

Exceptions

<i>std::range_error</i>	Value not between min and max
-------------------------	-------------------------------

Definition at line 82 of file [lock.hpp](#).

6.1.3.4 operator-=()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
void histogram_lock< max_elem, min_elem, T >::operator-= (
    T val ) [inline]
```

Decreases the position of the lock by val amount.

Parameters

<i>val</i>	Delta
------------	-------

Exceptions

<i>std::range_error</i>	Value not between min and max
-------------------------	-------------------------------

Definition at line 93 of file [lock.hpp](#).

6.1.3.5 operator==()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
void histogram_lock< max_elem, min_elem, T >::operator==(
    T val ) [inline]
```

Sets the current position of the lock.

Parameters

<i>val</i>	New position
------------	--------------

Exceptions

<i>std::range_error</i>	Value not between min and max
-------------------------	-------------------------------

Definition at line 73 of file [lock.hpp](#).

6.1.3.6 set_current_value()

```
template<std::size_t max_elem, std::size_t min_elem = 0, std::signed_integral T = int>
void histogram_lock< max_elem, min_elem, T >::set_current_value (
    T val ) [inline]
```

sets the current position of the lock

Parameters

<i>val</i>	New position
------------	--------------

Exceptions

<i>std::range_error</i>	Value not between min and max
-------------------------	-------------------------------

Definition at line 61 of file [lock.hpp](#).

The documentation for this class was generated from the following file:

- 2025/Day1/[lock.hpp](#)

6.2 input_parser< T > Class Template Reference

An iterator that goes converts file to signed integers.

```
#include <input_parser.hpp>
```

Classes

- struct [itor](#)

Public Member Functions

- [input_parser](#) (std::filesystem::path path)
- [itor begin](#) ()
- [itor end](#) ()

6.2.1 Detailed Description

```
template<std::signed_integral T>
class input_parser< T >
```

An iterator that goes converts file to signed integers.

Template Parameters

<i>T</i>	Type of integral that the iterator will generate
----------	--

Definition at line 39 of file [input_parser.hpp](#).

6.2.2 Constructor & Destructor Documentation

6.2.2.1 input_parser()

```
template<std::signed_integral T>
input_parser< T >::input_parser (
    std::filesystem::path path ) [inline]
```

Definition at line 43 of file [input_parser.hpp](#).

6.2.3 Member Function Documentation

6.2.3.1 begin()

```
template<std::signed_integral T>
itor input_parser< T >::begin ( ) [inline]
```

Definition at line 46 of file [input_parser.hpp](#).

6.2.3.2 end()

```
template<std::signed_integral T>
itor input_parser< T >::end ( ) [inline]
```

Definition at line 48 of file [input_parser.hpp](#).

The documentation for this class was generated from the following file:

- 2025/Day1/[input_parser.hpp](#)

6.3 input_parser< T >::itor Struct Reference

Public Types

- using [value_type](#) = T
- using [iterator_category](#) = std::input_iterator_tag
- using [difference_type](#) = std::ptrdiff_t

Public Member Functions

- [itor](#) (std::shared_ptr< std::ifstream > f)
- value_type [operator*](#) () const
- [itor](#) & [operator++](#) ()
- [itor](#) [operator++](#) (int)
- bool [operator==](#) (const [itor](#) &o) const
- bool [operator!=](#) (const [itor](#) &o) const

6.3.1 Detailed Description

```
template<std::signed_integral T>
struct input_parser< T >::itor
```

Definition at line 36 of file [input_parser.hxx](#).

6.3.2 Member Typedef Documentation

6.3.2.1 difference_type

```
template<std::signed_integral T>
using input_parser< T >::itor::difference_type = std::ptrdiff_t
```

Definition at line 39 of file [input_parser.hxx](#).

6.3.2.2 `iterator_category`

```
template<std::signed_integral T>
using input_parser< T >::itor::iterator_category = std::input_iterator_tag
```

Definition at line 38 of file `input_parser.hxx`.

6.3.2.3 `value_type`

```
template<std::signed_integral T>
using input_parser< T >::itor::value_type = T
```

Definition at line 37 of file `input_parser.hxx`.

6.3.3 Constructor & Destructor Documentation

6.3.3.1 `itor()` [1/2]

```
template<std::signed_integral T>
input_parser< T >::itor::itor ( ) [inline], [noexcept]
```

Definition at line 41 of file `input_parser.hxx`.

6.3.3.2 `itor()` [2/2]

```
template<std::signed_integral T>
input_parser< T >::itor::itor (
    std::shared_ptr< std::ifstream > f ) [inline], [explicit]
```

Definition at line 44 of file `input_parser.hxx`.

6.3.4 Member Function Documentation

6.3.4.1 `operator"!="()`

```
template<std::signed_integral T>
bool input_parser< T >::itor::operator!= (
    const itor & o ) const [inline]
```

Definition at line 81 of file `input_parser.hxx`.

6.3.4.2 operator*()

```
template<std::signed_integral T>
value_type input_parser< T >::itor::operator* ( ) const [inline]
```

Definition at line 49 of file [input_parser.hxx](#).

6.3.4.3 operator++() [1/2]

```
template<std::signed_integral T>
itor & input_parser< T >::itor::operator++ ( ) [inline]
```

Definition at line 51 of file [input_parser.hxx](#).

6.3.4.4 operator++() [2/2]

```
template<std::signed_integral T>
itor input_parser< T >::itor::operator++ (
    int ) [inline]
```

Definition at line 73 of file [input_parser.hxx](#).

6.3.4.5 operator==()

```
template<std::signed_integral T>
bool input_parser< T >::itor::operator==(
    const itor & o ) const [inline]
```

Definition at line 79 of file [input_parser.hxx](#).

The documentation for this struct was generated from the following file:

- [2025/Day1/input_parser.hxx](#)

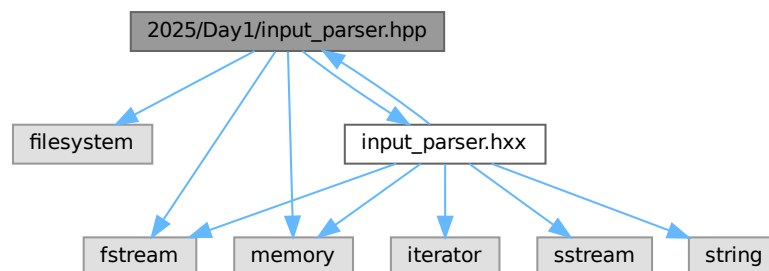
Chapter 7

File Documentation

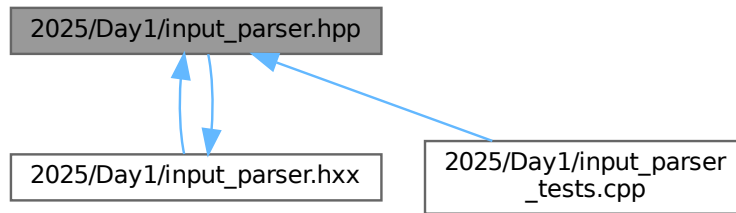
7.1 2025/Day1/input_parser.hpp File Reference

Functions to parse the input of the file, converting them to signed integers.

```
#include <filesystem>
#include <fstream>
#include <memory>
#include "input_parser.hxx"
Include dependency graph for input_parser.hpp:
```



This graph shows which files directly or indirectly include this file:



Classes

- class `input_parser< T >`
An iterator that goes converts file to signed integers.

7.1.1 Detailed Description

Functions to parse the input of the file, converting them to signed integers.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-14

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file `input_parser.hpp`.

7.2 input_parser.hpp

[Go to the documentation of this file.](#)

```

00001 /**
00002  * \file input_parser.hpp
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief Functions to parse the input of the file, converting them to signed
00005  * integers.
00006  * \version 0.1
00007  * \date 2026-05-14
00008  *
00009  * \copyright Copyright (c) 2026
00010  *
00011  * \parblock License:
00012  *
00013  * This program is free software: you can redistribute it and/or modify
00014  * it under the terms of the GNU General Public License as published by
00015  * the Free Software Foundation, either version 3 of the License, or
00016  * (at your option) any later version.
00017  *
00018  * This program is distributed in the hope that it will be useful,
00019  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00020  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00021  * GNU General Public License for more details.
00022  *
00023  * You should have received a copy of the GNU General Public License
00024  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00025  * \endparblock
00026  *
00027  */
00028
00029 #pragma once
00030 #include <filesystem>
00031 #include <fstream>
00032 #include <memory>
00033
00034 /**
00035  * \brief An iterator that goes converts file to signed integers
00036  * \tparam T Type of integral that the iterator will generate
00037  */
00038 template<std::signed_integral T>
00039 class input_parser {
00040 public:
00041     struct itor;
00042
00043     input_parser(std::filesystem::path path)
00044         : file_ptr(std::make_shared<std::ifstream>(path)) {}
00045
00046     itor begin() { return itor(file_ptr); }
00047
00048     itor end() { return itor(); }
00049
00050 private:
00051     std::shared_ptr<std::ifstream> file_ptr;
00052 };
00053
00054 #include "input_parser.hxx"

```

7.3 2025/Day1/input_parser.hxx File Reference

Implementation of the file parser iterator.

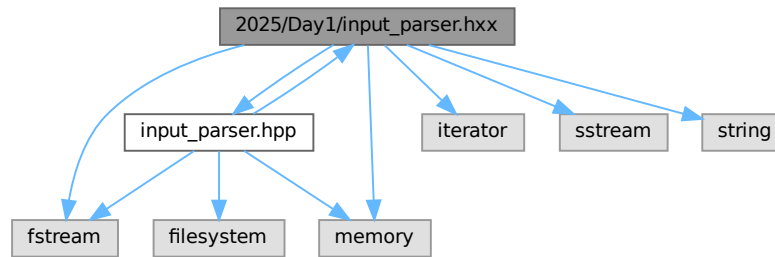
```

#include "input_parser.hpp"
#include <fstream>
#include <iterator>
#include <memory>
#include <sstream>

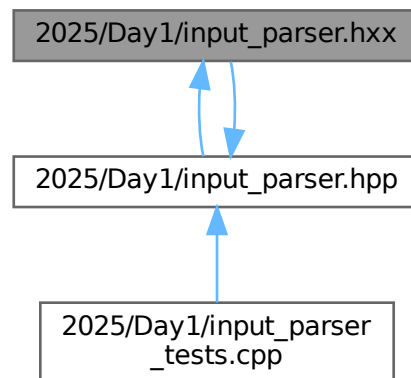
```

```
#include <string>
```

Include dependency graph for input_parser.hxx:



This graph shows which files directly or indirectly include this file:



Classes

- struct `input_parser< T >::itor`

7.3.1 Detailed Description

Implementation of the file parser iterator.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-14

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file [input_parser.hxx](#).

7.4 input_parser.hxx

[Go to the documentation of this file.](#)

```
00001 /**
00002  * \file input_parser.hxx
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief Implementation of the file parser iterator
00005  * \version 0.1
00006  * \date 2026-05-14
00007  *
00008  * \copyright Copyright (c) 2026
00009  *
00010  * \parblock License:
00011  *
00012  * This program is free software: you can redistribute it and/or modify
00013  * it under the terms of the GNU General Public License as published by
00014  * the Free Software Foundation, either version 3 of the License, or
00015  * (at your option) any later version.
00016  *
00017  * This program is distributed in the hope that it will be useful,
00018  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00019  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00020  * GNU General Public License for more details.
00021  *
00022  * You should have received a copy of the GNU General Public License
00023  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00024  * \endparblock
00025  *
```

```

00026  */
00027  #include "input_parser.hpp"
00028  #include <fstream>
00029  #include <iterator>
00030  #include <memory>
00031  #include <sstream>
00032  #include <string>
00033
00034  template<std::signed_integral T>
00035
00036  struct input_parser<T>::itor {
00037      using value_type      = T;
00038      using iterator_category = std::input_iterator_tag;
00039      using difference_type  = std::ptrdiff_t;
00040
00041      itor() noexcept
00042          : file_ptr(nullptr) {}
00043
00044      explicit itor(std::shared_ptr<std::ifstream> f)
00045          : file_ptr(f) {
00046          ++(*this);
00047      }
00048
00049      value_type operator*() const { return current; }
00050
00051      itor& operator++() {
00052          if (!file_ptr || !(*file_ptr)) {
00053              file_ptr.reset();
00054              return *this;
00055          }
00056
00057          std::string line;
00058          if (!std::getline(*file_ptr, line)) {
00059              file_ptr.reset();
00060              return *this;
00061          }
00062
00063          std::istringstream iss{ line };
00064          value_type          output = 0;
00065          char                c{};
00066          iss >> c;
00067          iss >> output;
00068          if (c == 'L') output *= -1;
00069          current = output;
00070          return *this;
00071      }
00072
00073      itor operator++(int) {
00074          itor tmp = *this;
00075          ++(*this);
00076          return tmp;
00077      }
00078
00079      bool operator==(const itor& o) const { return file_ptr == o.file_ptr; }
00080
00081      bool operator!=(const itor& o) const { return !(*this == o); }
00082
00083  private:
00084      std::shared_ptr<std::ifstream> file_ptr;
00085      value_type                    current{};
00086  };
00087
00088  static_assert(std::input_iterator<input_parser<int>::itor>,
00089              "Iterator is not correct");

```

7.5 2025/Day1/input_parser_tests.cpp File Reference

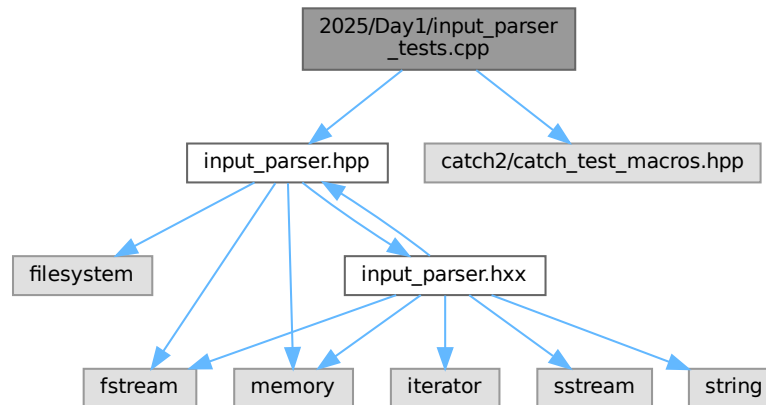
Tests the input parser.

```

#include "input_parser.hpp"
#include <catch2/catch_test_macros.hpp>

```

Include dependency graph for input_parser_tests.cpp:



Functions

- `TEST_CASE` ("Check inputs come out correctly", "[DayOne][Parser]")

7.5.1 Detailed Description

Tests the input parser.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-14

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file `input_parser_tests.cpp`.

7.5.2 Function Documentation

7.5.2.1 TEST_CASE()

```
TEST_CASE (
    "Check inputs come out correctly" ,
    "" [DayOne][Parser] )
```

Definition at line 31 of file [input_parser_tests.cpp](#).

7.6 input_parser_tests.cpp

[Go to the documentation of this file.](#)

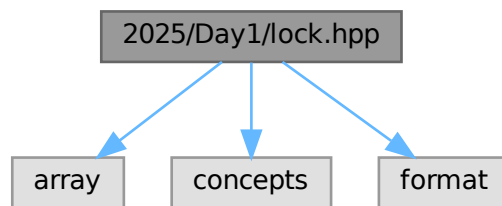
```
00001 /**
00002  * \file input_parser_tests.cpp
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief Tests the input parser
00005  * \version 0.1
00006  * \date 2026-05-14
00007  *
00008  * \copyright Copyright (c) 2026
00009  *
00010  * \parblock License:
00011  *
00012  *   This program is free software: you can redistribute it and/or modify
00013  *   it under the terms of the GNU General Public License as published by
00014  *   the Free Software Foundation, either version 3 of the License, or
00015  *   (at your option) any later version.
00016  *
00017  *   This program is distributed in the hope that it will be useful,
00018  *   but WITHOUT ANY WARRANTY; without even the implied warranty of
00019  *   MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00020  *   GNU General Public License for more details.
00021  *
00022  *   You should have received a copy of the GNU General Public License
00023  *   along with this program. If not, see <https://www.gnu.org/licenses/>.
00024  * \endparblock
00025  *
00026  */
00027
00028 #include "input_parser.hpp"
00029 #include <catch2/catch_test_macros.hpp>
00030
00031 TEST_CASE("Check inputs come out correctly", "[DayOne][Parser]") {
00032     int correct_vals[] = { -68, -30, 48, -5, 60, -55, -1, -99, 14, -82 };
00033
00034     std::filesystem::path tempDir = std::filesystem::temp_directory_path();
00035     std::filesystem::path tempFile = tempDir / "AVC.2025.1.Test.tmp";
00036
00037     std::ofstream file(tempFile);
00038     file << "L68\n"
00039           << "L30\n"
00040           << "R48\n"
00041           << "L5\n"
00042           << "R60\n"
00043           << "L55\n"
00044           << "L1\n"
00045           << "L99\n"
00046           << "R14\n"
00047           << "L82\n";
00048     file.close();
00049
00050     auto parser = input_parser<int>(tempFile).begin();
00051     for (auto i : correct_vals) {
00052         CHECK(*parser++ == i);
00053     }
00054     std::remove(tempFile.c_str());
00055 }
```


7.7 2025/Day1/lock.hpp File Reference

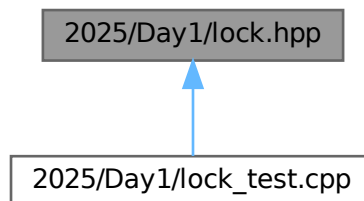
Implements a lock that holds a histogram of values.

```
#include <array>
#include <concepts>
#include <format>
```

Include dependency graph for lock.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- class `histogram_lock< max_elem, min_elem, T >`
A lock that keeps a histogram of how frequently each value is present.

7.7.1 Detailed Description

Implements a lock that holds a histogram of values.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-13

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file [lock.hpp](#).

7.8 lock.hpp

[Go to the documentation of this file.](#)

```
00001 /**
00002  * \file lock.hpp
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief Implements a lock that holds a histogram of values.
00005  * \version 0.1
00006  * \date 2026-05-13
00007  *
00008  * \copyright Copyright (c) 2026
00009  *
00010  * \parblock License:
00011  *
00012  * This program is free software: you can redistribute it and/or modify
00013  * it under the terms of the GNU General Public License as published by
00014  * the Free Software Foundation, either version 3 of the License, or
00015  * (at your option) any later version.
00016  *
00017  * This program is distributed in the hope that it will be useful,
```

```

00018 * but WITHOUT ANY WARRANTY; without even the implied warranty of
00019 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00020 * GNU General Public License for more details.
00021 *
00022 * You should have received a copy of the GNU General Public License
00023 * along with this program. If not, see <https://www.gnu.org/licenses/>.
00024 * \endparblock
00025 *
00026 */
00027 #pragma once
00028 #include <array>
00029 #include <concepts>
00030 #include <format>
00031
00032 /**
00033 * \brief A lock that keeps a histogram of how frequently each value is present
00034 *
00035 * \todo refactor the histogram into another class, and have it a separate
00036 * dependency from the lock
00037 *
00038 * @tparam max_elem The largest value on the lock
00039 * @tparam min_elem The smallest value on the lock
00040 * @tparam T The type of the values on the lock
00041 */
00042 template<std::size_t max_elem,
00043         std::size_t min_elem = 0,
00044         std::signed_integral T = int>
00045     requires (max_elem > min_elem)
00046     class histogram_lock {
00047     public:
00048         /**
00049          * \brief Construct a new lock object at position init
00050          *
00051          * \param init Value that the lock starts at
00052          */
00053         histogram_lock(T init) { set_current_value(init); }
00054
00055         /**
00056          * \brief sets the current position of the lock
00057          *
00058          * \param val New position
00059          * \throws std::range_error Value not between min and max
00060          */
00061         void set_current_value(T val) {
00062             bounds_check(val);
00063             current_value = std::move(val);
00064             histogram[val - min]++;
00065         }
00066
00067         /**
00068          * \brief Sets the current position of the lock
00069          *
00070          * \param val New position
00071          * \throws std::range_error Value not between min and max
00072          */
00073         void operator==(T val) { set_current_value(val); }
00074
00075         /**
00076          * \brief Increases the position of the lock by \ref val "val" amount
00077          *
00078          * \param val Delta
00079          * \throws std::range_error Value not between min and max
00080          */
00081
00082         void operator+=(T val) {
00083             // Add val to current_value, then wrap and set
00084             set_current_value(wrap_to_range(current_value + val));
00085         }
00086
00087         /**
00088          * \brief Decreases the position of the lock by \ref val "val" amount
00089          *
00090          * \param val Delta
00091          * \throws std::range_error Value not between min and max
00092          */
00093         void operator-=(T val) { this->operator+=(-val); }
00094
00095         /**
00096          * \brief Get the current value object
00097          */
00098         const T get_current_value() const { return current_value; }

```

```

00099
00100  /**
00101   * \brief Gets how many times the lock was at val
00102   *
00103   * \param val
00104   * \return count
00105   */
00106  T get_frequency(T val) {
00107      bounds_check(val);
00108      return histogram[val - min];
00109  }
00110
00111 private:
00112     static constexpr T min    = static_cast<T>(min_elem);
00113     static constexpr T max    = static_cast<T>(max_elem);
00114     static constexpr T range = max - min + 1;
00115
00116     std::array<T, max - min + 1> histogram{};
00117     T current_value;
00118
00119  /**
00120   * \brief Checks if the argument is in the range that this lock can handle
00121   *
00122   * Throws if out of bounds or returns true (can not return false).
00123   * \param t checked value
00124   * \return true
00125   * \throws std::range_error value not in range
00126   */
00127  bool bounds_check(T t) {
00128      if (t > max || t < min)
00129          throw std::range_error( std::format(
00130              "Value {} not in lock bounds [{} - {}]", t, min, max) );
00131      return true;
00132  }
00133
00134  /**
00135   * \brief Does the modular wrapping math.
00136   */
00137  static constexpr T wrap_to_range(T val) {
00138      if (val >= min && val <= max) [[likely]]
00139          return val;
00140      return min + ((val - min) % range + range) % range;
00141  }
00142  };

```

7.9 2025/Day1/lock_test.cpp File Reference

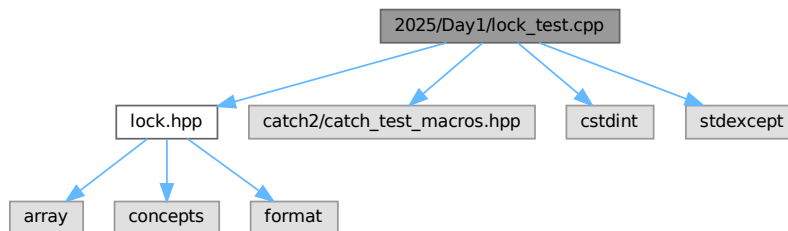
Tests the lock to see if it works as expected.

```

#include "lock.hpp"
#include <catch2/catch_test_macros.hpp>
#include <cstdint>
#include <stdexcept>

```

Include dependency graph for lock_test.cpp:



Functions

- [TEST_CASE](#) ("Ensure that Lock can be created with any integral type", "[DayOne][Lock][Template]")
- [TEST_CASE](#) ("Ensure that the lock can be rotated, and the count is correct", "[DayOne][Lock][Use]")
- [TEST_CASE](#) ("Ensure that the the lock can roll around max value", "[DayOne][Lock][Overflow]")

7.9.1 Detailed Description

Tests the lock to see if it works as expected.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-13

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file [lock_test.cpp](#).

7.9.2 Function Documentation

7.9.2.1 TEST_CASE() [1/3]

```
TEST_CASE (
    "Ensure that Lock can be created with any integral type" ,
    " " [DayOne] [Lock] [Template] )
```

Definition at line 32 of file [lock_test.cpp](#).

7.9.2.2 TEST_CASE() [2/3]

```
TEST_CASE (
    "Ensure that the lock can be rotated,
    and the count is correct" ,
    "" [DayOne] [Lock] [Use] )
```

Definition at line 41 of file [lock_test.cpp](#).

7.9.2.3 TEST_CASE() [3/3]

```
TEST_CASE (
    "Ensure that the the lock can roll around max value" ,
    "" [DayOne] [Lock] [Overflow] )
```

Definition at line 56 of file [lock_test.cpp](#).

7.10 lock_test.cpp

[Go to the documentation of this file.](#)

```
00001 /**
00002  * \file lock_test.cpp
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief Tests the lock to see if it works as expected
00005  * \version 0.1
00006  * \date 2026-05-13
00007  *
00008  * \copyright Copyright (c) 2026
00009  *
00010  * \parblock License:
00011  *
00012  * This program is free software: you can redistribute it and/or modify
00013  * it under the terms of the GNU General Public License as published by
00014  * the Free Software Foundation, either version 3 of the License, or
00015  * (at your option) any later version.
00016  *
00017  * This program is distributed in the hope that it will be useful,
00018  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00019  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00020  * GNU General Public License for more details.
00021  *
00022  * You should have received a copy of the GNU General Public License
00023  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00024  * \endparblock
00025  *
00026  */
00027 #include "lock.hpp"
00028 #include <catch2/catch_test_macros.hpp>
00029 #include <cstdint>
00030 #include <stdexcept>
00031
00032 TEST_CASE("Ensure that Lock can be created with any integral type",
00033     "[DayOne][Lock][Template]") {
00034     histogram_lock<10, 0, int>{ 0 };
00035     histogram_lock<100, 0, char>{ 0 };
00036     histogram_lock<10, 0, long long>{ 0 };
00037     // histogram_lock<10, 0, std::uint8_t>{ 0 }; //Not signed
00038     histogram_lock<10, 0, std::int_least16_t>{ 0 };
00039 }
00040
00041 TEST_CASE("Ensure that the lock can be rotated, and the count is correct",
00042     "[DayOne][Lock][Use]") {
00043     histogram_lock<100> lock{ 50 };
00044     REQUIRE(lock.get_current_value() == 50);
```

```

00045     REQUIRE(lock.get_frequency(0) == 0);
00046     REQUIRE(lock.get_frequency(50) == 1);
00047     lock += 5;
00048     lock -= 10;
00049     lock += 5;
00050     REQUIRE(lock.get_current_value() == 50);
00051     REQUIRE(lock.get_frequency(50) == 2);
00052     REQUIRE(lock.get_frequency(55) == 1);
00053     REQUIRE(lock.get_frequency(45) == 1);
00054 }
00055
00056 TEST_CASE("Ensure that the the lock can roll around max value",
00057     "[DayOne][Lock][Overflow]") {
00058     histogram_lock<99> lock{ 50 };
00059     REQUIRE_NOTHROW(lock += 120);
00060     REQUIRE_NOTHROW(lock -= 120);
00061     REQUIRE_THROWS_AS(lock.set_current_value(150), std::range_error);
00062     GIVEN("Lock at 50") {
00063         lock.set_current_value(50);
00064         WHEN("Incremented by 60") {
00065             lock += 60;
00066             THEN("Lock should be 10") {
00067                 REQUIRE(lock.get_current_value() == 10);
00068                 REQUIRE(lock.get_frequency(10) == 1);
00069             }
00070         }
00071         WHEN("Decrementd by 60") {
00072             lock -= 60;
00073             THEN("Lock should be 90") {
00074                 REQUIRE(lock.get_current_value() == 90);
00075                 REQUIRE(lock.get_frequency(90) == 1);
00076             }
00077         }
00078         WHEN("Incremented by 160") {
00079             lock += 160;
00080             THEN("Lock should be 10") {
00081                 REQUIRE(lock.get_current_value() == 10);
00082                 REQUIRE(lock.get_frequency(10) == 1);
00083             }
00084         }
00085         WHEN("Decrementd by 160") {
00086             lock -= 160;
00087             THEN("Lock should be 90") {
00088                 REQUIRE(lock.get_current_value() == 90);
00089                 REQUIRE(lock.get_frequency(90) == 1);
00090             }
00091         }
00092     }
00093 }

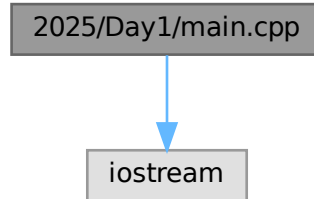
```

7.11 2025/Day1/main.cpp File Reference

My Day One Solution Problem.

```
#include <iostream>
```

Include dependency graph for main.cpp:



Functions

- int [main](#) (int argc, char const *argv[])

7.11.1 Detailed Description

My Day One Solution Problem.

Author

Dylan Shah (code@dylan-shah.com)

Version

0.1

Date

2026-05-13

Copyright

Copyright (c) 2026

License:

This program is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <https://www.gnu.org/licenses/>.

Definition in file [main.cpp](#).

7.11.2 Function Documentation

7.11.2.1 main()

```
int main (
    int argc,
    char const * argv[ ] )
```

Definition at line 31 of file [main.cpp](#).

7.12 main.cpp

[Go to the documentation of this file.](#)

```
00001 /**
00002  * \file main.cpp
00003  * \author Dylan Shah (code@dylan-shah.com)
00004  * \brief My Day One Solution
00005  * \ref https://adventofcode.com/2025/day/1 "Problem"
00006  * \version 0.1
00007  * \date 2026-05-13
00008  *
00009  * \copyright Copyright (c) 2026
00010  *
00011  * \parblock License:
00012  *
00013  * This program is free software: you can redistribute it and/or modify
00014  * it under the terms of the GNU General Public License as published by
00015  * the Free Software Foundation, either version 3 of the License, or
00016  * (at your option) any later version.
00017  *
00018  * This program is distributed in the hope that it will be useful,
00019  * but WITHOUT ANY WARRANTY; without even the implied warranty of
00020  * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
00021  * GNU General Public License for more details.
00022  *
00023  * You should have received a copy of the GNU General Public License
00024  * along with this program. If not, see <https://www.gnu.org/licenses/>.
00025  * \endparblock
00026  *
00027  */
00028
00029 #include <iostream>
00030
00031 int main(int argc, char const* argv[]) {
00032     std::cout << "test";
00033     return 0;
00034 }
```


Index

2025/Day1/input_parser.hpp, [27](#), [29](#)
2025/Day1/input_parser.hxx, [29](#), [31](#)
2025/Day1/input_parser_tests.cpp, [32](#), [34](#)
2025/Day1/lock.hpp, [35](#), [36](#)
2025/Day1/lock_test.cpp, [38](#), [40](#)
2025/Day1/main.cpp, [41](#), [43](#)

begin
 input_parser< T >, [23](#)

difference_type
 input_parser< T >::itor, [24](#)

end
 input_parser< T >, [23](#)

get_current_value
 histogram_lock< max_elem, min_elem, T >, [20](#)
get_frequency
 histogram_lock< max_elem, min_elem, T >, [20](#)
GNU GENERAL PUBLIC LICENSE, [3](#)

histogram_lock
 histogram_lock< max_elem, min_elem, T >, [20](#)
histogram_lock< max_elem, min_elem, T >, [19](#)
 get_current_value, [20](#)
 get_frequency, [20](#)
 histogram_lock, [20](#)
 operator+=, [21](#)
 operator-=, [21](#)
 operator==, [22](#)
 set_current_value, [22](#)

input_parser
 input_parser< T >, [23](#)

input_parser< T >, [23](#)
 begin, [23](#)
 end, [23](#)
 input_parser, [23](#)

input_parser< T >::itor, [24](#)
 difference_type, [24](#)
 iterator_category, [24](#)
 itor, [25](#)
 operator!=, [25](#)
 operator++, [26](#)
 operator==, [26](#)
 operator*, [25](#)

 value_type, [25](#)
input_parser_tests.cpp
 TEST_CASE, [34](#)
iterator_category
 input_parser< T >::itor, [24](#)
itor
 input_parser< T >::itor, [25](#)

lock_test.cpp
 TEST_CASE, [39](#), [40](#)

main
 main.cpp, [43](#)
main.cpp
 main, [43](#)

My solutions to the Advent Of Code problems., [1](#)

operator!=
 input_parser< T >::itor, [25](#)
operator++
 input_parser< T >::itor, [26](#)
operator+=
 histogram_lock< max_elem, min_elem, T >, [21](#)
operator-=
 histogram_lock< max_elem, min_elem, T >, [21](#)
operator==
 histogram_lock< max_elem, min_elem, T >, [22](#)
 input_parser< T >::itor, [26](#)
operator*
 input_parser< T >::itor, [25](#)

set_current_value
 histogram_lock< max_elem, min_elem, T >, [22](#)

TEST_CASE
 input_parser_tests.cpp, [34](#)
 lock_test.cpp, [39](#), [40](#)

Todo List, [13](#)

value_type
 input_parser< T >::itor, [25](#)